МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

Львівський національний університет імені Івана Франка

Факультет електроніки та комп'ютерних технологій

ЗВІТ

про виконання лабораторної роботи №5

«Обробка помилок, файли та серіалізація»

з дисципліни «Професійний Python»

Варіант №5 — «Каталог автомобілів»

Виконав:

студент групи ФЕП-32

Горин Михайло

Репозиторій: github.com/MykhailoG212/Prof_python

Львів — 2026

2. Номер і тема лабораторної роботи

Лабораторна робота №5. Тема: «Обробка помилок, файли та серіалізація». У звіті наведено модуль надійного імпорту та експорту каталогу автомобілів: зовнішня конфігурація, потокове читання CSV, валідація, контрольована обробка помилок, логування й атомарний JSON-експорт.

3. Мета роботи

Сформувати практичні навички створення надійних Python-програм, що коректно працюють із файловими ресурсами та некоректними зовнішніми даними. Застосувати exceptions, власну ієрархію помилок, exception chaining, context managers, pathlib, CSV, JSON, YAML-конфігурацію, logging, streaming I/O і атомарний запис файлів.

4. Номер індивідуального варіанта

Варіант №5 — «Каталог автомобілів». Наступний етап наскрізного проєкту використовує знайому предметну область і додає інфраструктурний модуль для контрольованого обміну даними між CSV-файлом та JSON-експортом.

5. Постановка задачі

Потрібно реалізувати імпорт та експорт каталогу автомобілів з полями vehicle_id, brand, model, year, price, mileage. Система має перевіряти додатний ID, непорожні марку й модель, допустимий діапазон року та ціни, невід'ємний пробіг. Вимоги включають streaming CSV import, JSON export, YAML-налаштування діапазонів року й ціни, статистику та logging.

5.1. Вимоги індивідуального варіанта №5

Вимога	Реалізація
Поля vehicle_id, brand, model, year, price, mileage	Vehicle dataclass і parse_vehicle() у reliable/models.py та reliable/importers.py.
Streaming CSV import	csv.DictReader у read_vehicles(); файл читається в межах with і рядки обробляються по одному.
Діапазони років і цін	config/lab5.yaml; load_config() перевіряє конфігурацію до початку обробки.
Strict і tolerant режими	invalid_record_policy визначає продовження або зупинку на першому некоректному записі.
JSON export і logging	export_json() виконує атомарний запис; services.py фіксує warning для невалідного рядка.

5.2. Загальні вимоги лабораторної роботи

Компонент	Виконання
Custom exceptions	ApplicationError, ConfigurationError, DataImportError, RecordValidationError, DataExportError.
Exception chaining	Перетворення ValueError/KeyError/OSError на domain error через raise ... from error.
Context managers	with path.open(), власний @contextmanager atomic_text_writer().
Безпечний output	Тимчасовий файл замінює ціль через os.replace() лише після успішного json.dump().

6. Структура проєкту

Нова підсистема не змінює попередні модулі каталогу. Вона відокремлює конфігурацію, ввід-вивід, доменні дані та orchestration, тому правила валідації й обробка файлових помилок не змішуються з UI або аналітикою.

```
src/car_catalog/reliable/
├── __init__.py
├── config.py        # YAML loader і fail-fast validation
├── exceptions.py    # hierarchy application errors
├── models.py        # Vehicle та config dataclasses
├── importers.py     # streaming CSV reader і parser
├── exporters.py     # atomic JSON writer
├── services.py      # strict/tolerant orchestration, statistics, logging
└── main.py          # CLI entry point
config/lab5.yaml
data/lab5_vehicles.csv
tests/test_reliable_importer.py
```

7. Архітектура модуля

Конфігурація є окремою точкою входу для параметрів. Вхідний CSV проходить потокове читання, парсинг і валідацію. Service застосовує політику strict або tolerant, збирає статистику та передає тільки valid записи експортеру. Вихідний JSON пишеться через власний context manager.

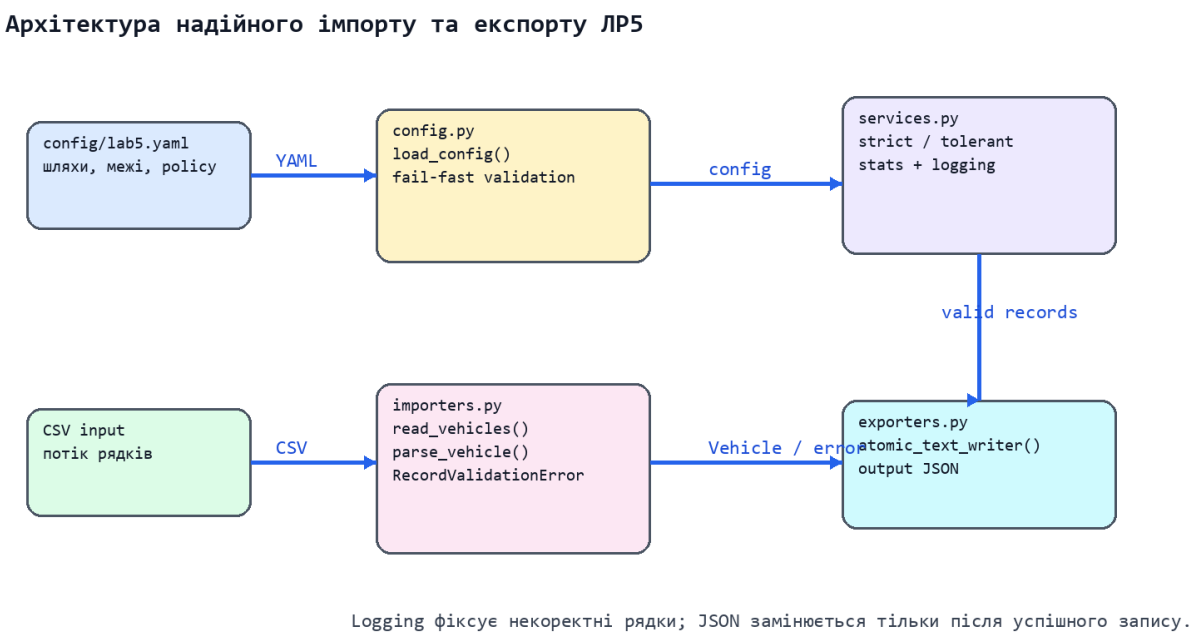


Рисунок 1 — Компоненти модуля надійного імпорту та експорту ЛР5

8. Схема вхідних даних і приклад CSV

Поле	Тип	Правило
vehicle_id	int	Додатне ціле число.
brand, model	str	Непорожні значення після trim.
year	int	У межах minimum_year..maximum_year із YAML.
price	float	Скінченне число у межах minimum_price..maximum_price.
mileage	int	Невід'ємне ціле число.

```
vehicle_id,brand,model,year,price,mileage
1,Tesla,Model Y,2023,50000,12000
2,Toyota,Camry,2020,25000,45000
invalid,Unknown,Model,2020,not-a-price,100
3,BMW,M3,2022,72000,24000
```

9. Ієрархія власних винятків

Ієрархія відокремлює критичні помилки застосунку від помилок одиничних записів. Це дає змогу main.py завершитися контрольовано, а tolerant-режиму продовжити обробку після RecordValidationError.

```
"""Предметно-орієнтована ієрархія помилок ЛР5."""


class ApplicationError(Exception):
    """Базова помилка застосунку."""


class ConfigurationError(ApplicationError):
    """Конфігурація відсутня або містить недопустимі значення."""


class DataError(ApplicationError):
    """Базова помилка зовнішніх даних."""
```

```
class DataImportError(DataError):
    """Вхідний файл неможливо прочитати або розібрати."""


class RecordValidationError(DataError):
    """Один CSV-запис не відповідає схемі каталогу."""

    def __init__(self, message: str, *, line_number: int, field: str | None = None) -> None:
        super().__init__(message)
        self.line_number = line_number
        self.field = field


class DataExportError(DataError):
    """Вихідний JSON не вдалося безпечно записати."""
```

10. YAML configuration і fail-fast validation

Конфігурація зчитується перед відкриттям CSV. load_config() перевіряє обов'язкові ключі, policy, порядок меж року й ціни та наявність input file. Неправильне налаштування перетворюється на ConfigurationError із збереженням первинної причини.

```
input:
  path: data/lab5_vehicles.csv
output:
  path: output/vehicles.json
processing:
  minimum_year: 1886
  maximum_year: 2026
  minimum_price: 0
  maximum_price: 1000000
  invalid_record_policy: tolerant
logging:
  level: INFO
  path: logs/lab5.log


        result[section][key.strip()] = _scalar(raw_value)
    return result


def load_config(path: Path) -> AppConfig:
    """Повертає перевірену конфігурацію або завершує роботу до імпорту."""
    raw = _load_simple_yaml(path)
    try:
        processing = raw["processing"]
        policy = str(processing["invalid_record_policy"])
        if policy not in {"strict", "tolerant"}:
            raise ValueError("invalid_record_policy must be strict or tolerant")
        config = AppConfig(
            input_path=Path(str(raw["input"]["path"])),
            output_path=Path(str(raw["output"]["path"])),
            processing=ProcessingConfig(
                minimum_year=int(processing["minimum_year"]),
                maximum_year=int(processing["maximum_year"]),
                minimum_price=float(processing["minimum_price"]),
                maximum_price=float(processing["maximum_price"]),
                invalid_record_policy=cast("Any", policy),
            ),
            logging=LoggingConfig(
                level=str(raw["logging"]["level"]).upper(),
                path=Path(str(raw["logging"]["path"])),
            ),
        )
```

```
    except (KeyError, TypeError, ValueError) as error:
        raise ConfigurationError("Invalid YAML configuration") from error
    if config.processing.minimum_year > config.processing.maximum_year:
        raise ConfigurationError("minimum_year must not exceed maximum_year")
    if config.processing.minimum_price > config.processing.maximum_price:
```

11. Streaming reader і валідація

read_vehicles() використовує csv.DictReader усередині with. Генератор повертає результат для кожного рядка, не накопичуючи весь файл у пам’яті. parse_vehicle() розділяє перетворення типів і предметні правила; низькорівневі ValueError та KeyError передаються як explicit exception chain.

```
"""Потоковий CSV-імпорт із чіткими помилками валідації."""

import csv
import math
from collections.abc import Iterator
from pathlib import Path

from car_catalog.reliable.exceptions import DataImportError, RecordValidationError
from car_catalog.reliable.models import ProcessingConfig, Vehicle

_FIELDS = ("vehicle_id", "brand", "model", "year", "price", "mileage")


def parse_vehicle(row: dict[str, str | None], line_number: int, config: ProcessingConfig) ->
Vehicle:
    """Перетворює CSV-рядок на Vehicle, зберігаючи першопричину через chaining."""
    try:
        values = {name: (row[name] or "").strip() for name in _FIELDS}
    except KeyError as error:
        raise RecordValidationError("missing required column", line_number=line_number) from
error
    try:
        vehicle_id = int(values["vehicle_id"])
        year = int(values["year"])
        price = float(values["price"])
        mileage = int(values["mileage"])
    except ValueError as error:
        raise RecordValidationError("numeric field has invalid value", line_number=line_number)
from error
    if vehicle_id <= 0:
        raise RecordValidationError("vehicle_id must be positive", line_number=line_number,
field="vehicle_id")
    if not values["brand"] or not values["model"]:
        raise RecordValidationError("brand and model must be non-empty",
line_number=line_number)
    if not config.minimum_year <= year <= config.maximum_year:
        raise RecordValidationError("year is outside configured range",
line_number=line_number, field="year")
    if not math.isfinite(price) or not config.minimum_price <= price <= config.maximum_price:
        raise RecordValidationError("price is outside configured range",
line_number=line_number, field="price")
    if mileage < 0:
        raise RecordValidationError("mileage must be non-negative", line_number=line_number,
field="mileage")
    return Vehicle(vehicle_id, values["brand"], values["model"], year, price, mileage)


def read_vehicles(path: Path, config: ProcessingConfig) -> Iterator[tuple[int, Vehicle |
RecordValidationError]]:
    """Читає CSV рядок за рядком, тому файл не накопичується в пам'яті."""
    try:
        with path.open("r", encoding="utf-8", newline="") as source:
            reader = csv.DictReader(source)
```

```
            if reader.fieldnames is None or set(_FIELDS) - set(reader.fieldnames):
                raise DataImportError("CSV has no required header")
            for line_number, row in enumerate(reader, start=2):
                try:
                    yield line_number, parse_vehicle(row, line_number, config)
                except RecordValidationError as error:
                    yield line_number, error
    except OSError as error:
        raise DataImportError(f"Cannot read input CSV: {path}") from error
    except csv.Error as error:
        raise DataImportError("Malformed CSV input") from error
```

12. Strict і tolerant processing

У tolerant-режимі service збільшує invalid_count, пише warning у лог і переходить до наступного рядка. У strict-режимі той самий RecordValidationError стає причиною DataImportError, тому експорт не запускається. Це гарантує, що strict результат не містить частково обробленого набору.

```
def import_catalog(config: AppConfig) -> ImportReport:
    """Імпортує каталог за політикою strict або tolerant та експортує valid записи."""
    logger = configure_logging(config.logging.path, config.logging.level)
    report = ImportReport()
    for line_number, result in read_vehicles(config.input_path, config.processing):
        if isinstance(result, RecordValidationError):
            report.invalid_count += 1
            logger.warning("line=%s field=%s error=%s", line_number, result.field, result)
            if config.processing.invalid_record_policy == "strict":
                raise DataImportError(f"Import stopped at line {line_number}: {result}") from
result
            continue
        report.valid_count += 1
        report.vehicles.append(result)
    export_json(
        config.output_path,
        {
            "statistics": {"valid": report.valid_count, "invalid": report.invalid_count},
            "vehicles": [vehicle.to_dict() for vehicle in report.vehicles],
        },
    )
    logger.info("Import completed: valid=%s invalid=%s", report.valid_count,
report.invalid_count)
    return report
```

13. JSON serialization та атомарний export

atomic_text_writer() створює temporary file у тій самій теці, щоб os.replace() був атомарною операцією в межах файлової системи. Якщо json.dump() або запис завершився помилкою, finally видаляє temporary file, а старий output лишається незмінним.

```
"""JSON-експорт з атомарною заміною завершеного файла."""

import json
import os
from contextlib import contextmanager
from pathlib import Path
```

```
from tempfile import NamedTemporaryFile
from typing import Iterator, TextIO, cast

from car_catalog.reliable.exceptions import DataExportError


@contextmanager
def atomic_text_writer(path: Path) -> Iterator[TextIO]:
    """Створює тимчасовий файл і замінює ціль лише після успішного запису."""
    path.parent.mkdir(parents=True, exist_ok=True)
    temporary: Path | None = None
    try:
        with NamedTemporaryFile("w", encoding="utf-8", delete=False, dir=path.parent,
suffix=".tmp") as file:
            temporary = Path(file.name)
            yield cast(TextIO, file)
        os.replace(temporary, path)
    except OSError as error:
        raise DataExportError(f"Cannot export JSON to: {path}") from error
    finally:
        if temporary is not None and temporary.exists():
            temporary.unlink()


def export_json(path: Path, payload: dict[str, object]) -> None:
    try:
        with atomic_text_writer(path) as target:
            json.dump(payload, target, ensure_ascii=False, indent=2)
            target.write("\n")
    except (TypeError, ValueError) as error:
        raise DataExportError("JSON payload is not serializable") from error
```

14. Власний context manager

Власний context manager реалізовано декоратором @contextmanager. Він централізує lifecycle temporary file, успішну заміну осередкового файла та cleanup у finally. Тест test_atomic_writer_preserves_existing_file_after_failure підтверджує, що при штучному RuntimeError попередній output.json залишається зі старим вмістом.

15. Демонстраційний запуск

Для запуску використано config/lab5.yaml і data/lab5_vehicles.csv. У вихідному наборі є три коректні автомобілі та один запис з нечисловим vehicle_id і ціною. За tolerant policy програма експортує три valid записи, реєструє один invalid запис і завершується успішно.



Рисунок 2 — Результат запуску демонстраційного сценарію ЛР5

16. Logging і приклад повідомлення

Лог записується в logs/lab5.log у кодуванні UTF-8. Повідомлення містить номер рядка, поле за наявності та опис помилки, але не зберігає весь CSV-запис. Це дає достатній контекст для діагностики без дублювання даних у журналі.

```
2026-10-27 20:00:00,000 WARNING line=4 field=None error=numeric field has invalid value
2026-10-27 20:00:00,000 INFO Import completed: valid=3 invalid=1
```

17. Автоматизоване тестування

Новий tests/test_reliable_importer.py перевіряє tolerant import і JSON payload, зупинку strict import із збереженим exception cause, fail-fast конфігурацію та атомарний запис. Увесь набір проєкту включає також тести попередніх лабораторних робіт.



Рисунок 3 — Фактичний результат повного прогону pytest

18. Статична перевірка типів

Модулі ЛР5 мають анотації типів для конфігурації, domain model, ітераторів, context manager та сервісу. Окрема перевірка mypy охоплює новий пакет і тестовий модуль.



Рисунок 4 — Результат перевірки mypy для ЛР5

19. Результати strict і tolerant experiment

Показник	Tolerant	Strict
Коректні записи до помилки	3 exported	1 до першої помилки
Некоректний рядок	warning у log, обробка продовжується	DataImportError, процес зупиняється
JSON output	Містить лише 3 valid записи	Не створюється
Причина помилки	RecordValidationError врахований у статистиці	RecordValidationError доступний як __cause__

20. Фрагмент тестів ЛР5

```
"""Перевірки надійного CSV → JSON конвеєра ЛР5."""
```

```
import json
from pathlib import Path

import pytest

from car_catalog.reliable.config import load_config
from car_catalog.reliable.exceptions import ConfigurationError, DataImportError
from car_catalog.reliable.exporters import atomic_text_writer
from car_catalog.reliable.services import import_catalog


def write_config(path: Path, *, policy: str = "tolerant") -> Path:
    input_path = path.parent / "vehicles.csv"
    output_path = path.parent / "output" / "vehicles.json"
    log_path = path.parent / "logs" / "lab5.log"
    path.write_text(
        "\n".join(
            [
                "input:", f"  path: {input_path.as_posix()}",
                "output:", f"  path: {output_path.as_posix()}",
                "processing:", "  minimum_year: 1886", "  maximum_year: 2026",
                "  minimum_price: 0", "  maximum_price: 1000000",
                f"  invalid_record_policy: {policy}",
                "logging:", "  level: INFO", f"  path: {log_path.as_posix()}",
            ]
        ),
        encoding="utf-8",
    )
    return input_path


def write_csv(path: Path) -> None:
    path.write_text(
        "vehicle_id,brand,model,year,price,mileage\n"
        "1,Tesla,Model Y,2023,50000,12000\n"
        "bad,Toyota,Camry,2020,25000,30000\n"
        "2,BMW,M3,2022,72000,24000\n",
        encoding="utf-8",
    )


def test_tolerant_import_exports_only_valid_records_and_logs_error(tmp_path: Path) -> None:
    config_path = tmp_path / "config.yaml"
    input_path = write_config(config_path)
    write_csv(input_path)

    config = load_config(config_path)
    report = import_catalog(config)

    assert (report.valid_count, report.invalid_count) == (2, 1)
    payload = json.loads(config.output_path.read_text(encoding="utf-8"))
    assert payload["statistics"] == {"valid": 2, "invalid": 1}
    assert [vehicle["vehicle_id"] for vehicle in payload["vehicles"]] == [1, 2]
    assert "numeric field has invalid value" in config.logging.path.read_text(encoding="utf-8")


def test_strict_import_stops_and_does_not_create_output(tmp_path: Path) -> None:
    config_path = tmp_path / "config.yaml"
    input_path = write_config(config_path, policy="strict")
    write_csv(input_path)
    config = load_config(config_path)

    with pytest.raises(DataImportError, match="line 3") as error:
        import_catalog(config)

    assert error.value.__cause__ is not None
    assert not config.output_path.exists()


def test_invalid_config_fails_before_processing(tmp_path: Path) -> None:
    config_path = tmp_path / "broken.yaml"
    config_path.write_text("processing:\n  minimum_year: 2025\n", encoding="utf-8")
```

```
    with pytest.raises(ConfigurationError, match="Invalid YAML"):
        load_config(config_path)


def test_atomic_writer_preserves_existing_file_after_failure(tmp_path: Path) -> None:
    target = tmp_path / "output.json"
    target.write_text("old", encoding="utf-8")

    with pytest.raises(RuntimeError):
        with atomic_text_writer(target) as target_file:
            target_file.write("new")
            raise RuntimeError("export failure")

    assert target.read_text(encoding="utf-8") == "old"
    assert list(tmp_path.glob("*.tmp")) == []
```

21. Відповіді на контрольні запитання

1. Чому не слід перехоплювати bare except? Він приховує системні й програмні помилки; тут обробляються конкретні OSError, csv.Error, ValueError та domain exceptions.

2. Навіщо exception chaining? Воно зберігає технічну причину, коли ValueError або OSError перетворюється на зрозумілу помилку предметної області.

3. Чому використано with? Контекстний менеджер закриває CSV та temporary file навіть при exception.

4. Чому CSV читається потоково? DictReader повертає записи по одному, тому пам'ять не залежить від повного розміру input.

5. Для чого atomic export? Він не дає пошкодити наявний JSON, коли серіалізація або запис перериваються.

6. Коли застосовувати strict policy? Коли хоча б один некоректний запис робить весь набір недостовірним.

7. Коли застосовувати tolerant policy? Коли valid записи можна безпечно опрацювати, а помилки потрібно зафіксувати для подальшого виправлення.

8. Чому конфігурацію перевіряють на старті? Fail-fast запобігає довгій обробці з неправильними шляхами або межами.

22. Висновок

У лабораторній роботі реалізовано надійний модуль імпорту й експорту каталогу автомобілів для варіанта №5. Модуль отримує правила з YAML, потоково читає CSV, валідує кожний запис, контролює поведінку через strict/tolerant policy, веде журнал і створює JSON лише через атомарний output. Власні винятки та exception chaining роблять помилки зрозумілими й

діагностованими, а тести підтверджують очікувану поведінку для успішних і аварійних сценаріїв.